
```
{'player': playerChoice, 'computer': computerChoice}
```

In the `__init__()` method ❶, we create `ImageCollection` objects for the player and computer, each containing the rock, paper, and scissors images. In the `enter()` method ❷, we use the `replace()` method of `ImageCollection` ❸ to show the images that represent the player's and the computer's choices.

Then, the evaluation is quite simple ❹. If the computer and the player made the same choice, we have a tie, and we play an appropriate tie sound. If the player wins, we increment the player's score and play a happy sound. If the computer wins, we increment the computer's score and play a sad sound. We update the player's or the computer's score and show the scores in the matching text display fields.

After the `enter()` method runs (one time for each round), the `handleInputs()` method ❺ is called in every frame by the scene manager. When the user clicks Restart, we call the inherited `goToScene()` method to transfer back to the Play scene.

The `draw()` method ❻ draws everything in the window for this scene.

In this scene, we don't do any additional work in each frame, so we don't need to write an `update()` method. When the scene manager calls `update()`, the inherited method in the `Scene` base class runs and just executes a `pass` statement.

Communication Between Scenes

The scene manager provides a set of methods that allow scenes to communicate with each other by sending or requesting information. This communication won't be needed by all programs but can be highly useful. The scene manager allows any scene to:

- Request information from another scene
- Send information to another scene
- Send information to all other scenes